

PHASE 7

TESTING AND VALIDATION

*Proving the complete, end-to-end experience actually works — functionally, securely, visually,
and under real usage — before go-live.*

Employee Raise Issue — ServiceNow Implementation Guide

Standalone Phase Document 7 of 7

Document Version 1.0

About This Document

This document is a standalone, self-contained implementation guide for Phase 7 — Testing and Validation — of the Employee Raise Issue project. It can be read and actioned on its own, without needing the full multi-phase master documentation, though it references the wider project for context where useful.

Field	Detail
Application	Employee Raise Issue (built within the Employee Center scope)
Phase	Phase 7 of 7
Primary Owner	QA / Test Lead, with Platform Administrator support
Related Document	Employee Raise Issue Documentation (full 7-phase project reference)
Instance Referenced	dev218486.service-now.com
Document Version	1.0

Contents

1. Purpose and Scope
2. Where This Phase Fits in the Project
3. Objective
4. Prerequisites
 - 4.1 Test Environment Reference
5. Test Strategy Overview
6. Functional Testing
7. Screenshot Reference
8. Security Testing
9. UI/UX and Cross-Device Testing
10. Performance Testing
11. Defects Identified and Resolved
12. User Acceptance Testing (UAT)
13. Sign-Off Criteria
14. Common Issues and Troubleshooting
15. Best Practices
 - 15.1 Frequently Asked Questions
 - 15.2 Testing Terminology Reference
16. Summary and Project Wrap-Up
 - 16.1 Phase 7 Deliverables Checklist

1. Purpose and Scope

This document is the complete, standalone implementation guide for Phase 7 of the Employee Raise Issue project: testing and validating everything built in Phases 1 through 6, from an employee's first click through to a correctly routed, correctly populated record, before the solution is considered ready for production use.

Unlike Phases 1–6, this phase does not build a new component — it proves that the components already built work correctly, together, under realistic conditions. It assumes all six prior phases are complete on the instance being tested.

2. Where This Phase Fits in the Project

Phase 7 is the final technical phase of the seven covered across this set of documents. It draws on every earlier phase directly:

Phase	What Phase 7 Verifies About It
1	The application scope is correctly isolated and selectable
2	The table, fields, and choice values behave and validate as designed
3	UI Policies, dependencies, and their server-side mirrors all fire correctly
4	The Record Producer is discoverable and produces a correctly populated record
5	The portal, its pages, and navigation all work for a real employee
6	Widgets render correctly, filter data correctly, and perform acceptably

Because this phase depends on all six others, it is also the point where problems introduced in an earlier phase are most likely to surface for the first time — a routing bug from Phase 3, for example, may not be obvious until a real end-to-end submission is traced through in Phase 7.

3. Objective

Before this solution is promoted beyond the development instance, every configuration element from Phases 1–6 should be validated individually and then as an end-to-end flow, covering functional correctness, security, user experience, and performance.

4. Prerequisites

- Phases 1–6 complete on the instance being tested.
- At least two distinct non-admin test user accounts, in different departments, to properly exercise record-level access control.
- A short list of representative employees available for User Acceptance Testing (section 12).
- Agreement on the sign-off criteria (section 13) with whoever will approve production deployment.
- A test plan document or spreadsheet to track test case execution, pass/fail status, and any defects raised, separate from this guide.

4.1 Test Environment Reference

Item	Detail
Instance under test	dev218486.service-now.com (or the equivalent Test instance once promoted)
Portal URL	/employee_request (or the portal's own URL suffix from Phase 5)
Test User A	Standard employee, Department X — used for TC-01 through TC-05, TC-08
Test User B	Standard employee, Department Y — used for TC-06 (cross-user access check)
Test Admin	Administrator account — used only to verify configuration, never as the primary functional/security test identity

5. Test Strategy Overview

Testing in this phase is organized into four categories, each targeting a different kind of risk:

Category	What It Catches	Who Performs It
Functional	Broken logic, wrong field behavior, incorrect routing	QA / Test Lead
Security	Data exposure, access control gaps, bypassable validation	QA / Test Lead with Platform Administrator
UI/UX and Cross-Device	Confusing wording, broken layouts, unusable mobile experience	QA / Test Lead, ideally with a non-technical reviewer
Performance	Slow page loads, inefficient queries as data grows	Platform Administrator

NOTE: Run these roughly in the order presented: functional correctness first (there is little point security-testing a feature that does not work), then security, then UI/UX polish, then performance under load.

6. Functional Testing

#	Test Case	Expected Result
TC-01	Submit an issue with a category requiring an attachment and no attachment	Client blocks submission requesting an attachment; if bypassed, server-side Business Rule rejects the insert.
TC-02	Select a Category value	Subcategory choice list refreshes to show only that category's values.
TC-03	Set Priority = Critical	Preferred Contact Method field becomes visible and mandatory.
TC-04	Submit a complete, valid issue as a standard employee	Record is created with State = New, correct Assignment Group, and the employee is redirected to a confirmation page showing the issue number.
TC-05	Log in as the submitting employee and open My Issues	The newly submitted issue appears in the list with the correct priority badge and state.
TC-06	Log in as a different employee and open My Issues	The other employee's issue is not visible (record-level ACL enforced).
TC-07	Attempt to access the portal as an	User is redirected to the login page.

#	Test Case	Expected Result
	unauthenticated guest	
TC-08	Search the portal for a plain-language term like "issue" or "help"	The Raise Employee Issue catalog item appears in results.

7. Screenshot Reference

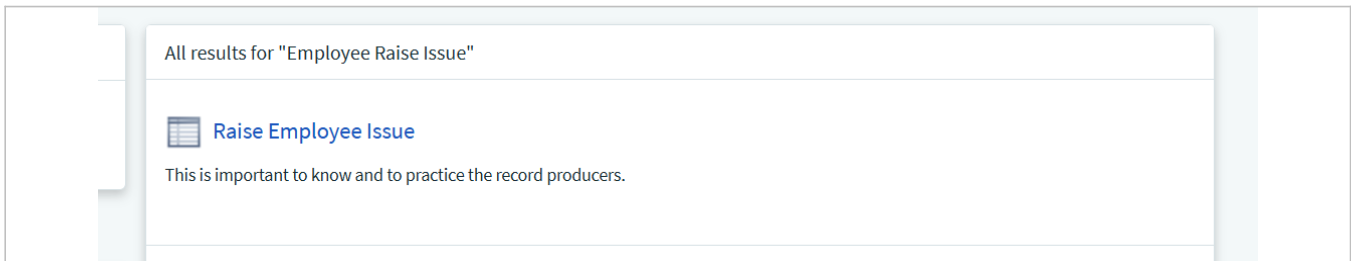


Figure 11 — Portal search successfully returning the Raise Employee Issue catalog item for the query "Employee Raise Issue."

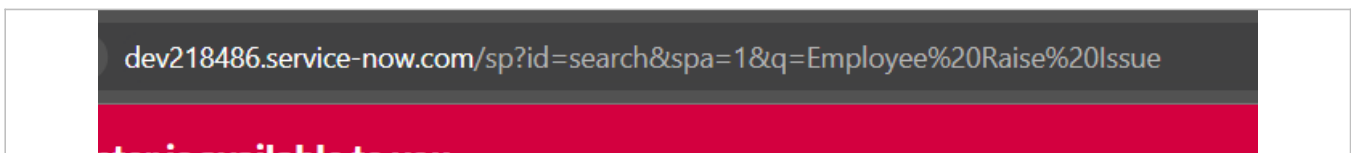


Figure 12 — Search results URL confirming the catalog item is discoverable via the portal's built-in search.

These two figures together satisfy TC-08 above: the item is not only configured correctly but is genuinely discoverable through the same search box a real employee would use.

8. Security Testing

Check	What to Confirm
Cross-user record access	Non-admin users cannot read or write Employee Issue records belonging to other employees, via the portal or direct table API access.
Record Producer create scope	The Record Producer's Create access allows submission for standard employees without granting broader table access.
Client-callable script exposure	Any script accessible to the client exposes no server-side logic beyond what is required.
Widget query scoping	The My Issues List widget's server-side query (Phase 6) filters strictly by the logged-in user and cannot be tricked into returning another user's records via a manipulated request.
Field-level ACL enforcement	Fields not intended for employee eyes (e.g. Resolution Notes) are genuinely inaccessible, not merely hidden from the default view.

WATCH OUT: Security testing should always include an attempt to submit or query data using the platform's REST API directly, not only through the portal UI. Client-side and even catalog-level protections do not apply to a direct API call — only table-level ACLs and Business Rules do.

9. UI/UX and Cross-Device Testing

- Verify the Raise an Issue page renders correctly on desktop, tablet, and mobile breakpoints.
- Verify UI Policies and Catalog Client Scripts behave identically on the Service Portal and, as a fallback, in the classic Service Catalog view.
- Verify all form validation messages are clear, employee-friendly, and free of internal field names or technical jargon.
- Verify the My Issues and Employee Issue Home widgets built in Phase 6 render cleanly at common mobile screen widths.
- Verify color contrast and keyboard navigation against the accessibility checklist established in Phase 5.

10. Performance Testing

- Measure page load time for Employee Issue Home and Raise an Issue under typical conditions, confirming both render within acceptable thresholds.
- Confirm the My Issues List widget's server script executes efficiently even as issue volume grows, using an indexed query on the requester field.
- Load a realistic volume of test records (for example, 200–500) into the Employee Issue table and re-time the My Issues page to confirm performance does not degrade sharply.

11. Defects Identified and Resolved

The table below is a working template for logging defects found during this phase; example entries show the kind of detail expected.

ID	Issue Found	Resolution
DEF-01	Subcategory retained a stale value after Category was changed a second time.	Updated the onChange Client Script to explicitly clear Subcategory on every category change, not only the first.
DEF-02	Attachment-mandatory rule did not reverse when Category was changed away from the triggering value.	Enabled Reverse if false on the associated UI Policy action.
DEF-03	My Issues page displayed a blank screen (instead of an empty-state message) for new employees.	Added explicit empty-state handling in the widget's client script and HTML template.
DEF-04	(template row — add new defects here)	(template row — add resolution here)

12. User Acceptance Testing (UAT)

A small group of representative employees, spanning different departments, should complete the end-to-end flow — from Portal Home through issue submission to viewing their submission in My Issues — and confirm the experience is intuitive without additional instructions.

1. Select 3–5 employees from different departments and, ideally, different levels of technical comfort.
2. Ask each to raise a real or realistic issue without being coached through the steps.
3. Observe (without prompting) where they hesitate, misread a label, or ask a question.
4. Collect feedback on wording, category names, and whether the confirmation and My Issues status were clear.

5. Incorporate feedback into catalog variable questions, confirmation page copy, or category naming before go-live.

13. Sign-Off Criteria

Before promoting this solution beyond the development or test instance, confirm each of the following:

- All functional test cases in section 6 pass without exception.
- No open High or Critical severity defects remain from section 11.
- Security testing (section 8) confirms record-level access is correctly scoped to the submitting employee and assigned support groups, including via direct API access.
- UAT participants (section 12) confirm the flow is understandable without training.
- Performance testing (section 10) shows acceptable load times under a realistic data volume.

Treat these as a gate, not a checklist to rush through — a solution that fails any of them is not yet ready for production, however close the go-live date.

14. Common Issues and Troubleshooting

Issue	Likely Cause	Resolution
A test case passes for an admin tester but fails for a real employee	Testing was performed with an account that has broader access than a standard employee.	Always run functional and security test cases from a genuine non-admin test account, never an administrator's own login.
Security test for cross-user data access is inconclusive	Both test accounts belong to the same assignment group or department, masking the issue.	Use two test employees in clearly different departments and, if applicable, different assignment groups.
Performance appears fine in testing but is slow in production	Test data volume was far lower than real production volume.	Load a realistic volume of test records before drawing performance conclusions, not just a handful.
UAT feedback conflicts between participants	Different employees have different mental models of the same workflow.	Look for the majority pattern and prioritize fixes that help the most common point of confusion, rather than chasing every individual preference.

15. Best Practices

BEST PRACTICE: Test as the least-privileged real-world user first, and only later as an administrator — this ordering catches ACL and role issues that admin-first testing consistently misses.

BEST PRACTICE: Treat a passing test suite as necessary, not sufficient — pair it with UAT from real employees, since testers who built the solution unconsciously already know how to use it correctly.

BEST PRACTICE: Keep the defect log (section 11) even after go-live; it becomes a valuable reference the next time a similar issue resurfaces months later.

15.1 Frequently Asked Questions

How much test data is "realistic" for performance testing?

There is no universal number — base it on a reasonable estimate of submission volume over the solution's first three to six months. For an organization of a few hundred employees, a few hundred test records is a reasonable starting point; scale up for larger populations or if issues are expected to be frequent (for example, daily IT requests).

Should UAT participants know they are testing, or should it be a blind test?

A short-notice, lightly-briefed test ("try raising a sample issue, we're checking the process is clear") tends to surface more genuine confusion than a fully explained walkthrough, since employees using the real system later will not have that context either.

What if sign-off criteria cannot all be met by the planned go-live date?

Treat the criteria in section 13 as a genuine gate, not a formality — it is far cheaper to delay go-live briefly than to launch with a known security or data-integrity gap. Where a criterion cannot be met in time, document the specific gap and get explicit, informed sign-off on the residual risk from whoever owns that decision.

Who should own the defect log after go-live?

Typically the Platform Administrator identified in the project's support model, since new defects reported after launch usually surface through the same support channels as ordinary issue tickets.

15.2 Testing Terminology Reference

Term	Meaning in This Context
Functional testing	Confirms features behave as designed — correct logic, correct data, correct routing.
Security testing	Confirms access controls hold up, including against direct API calls that bypass the UI.
UAT (User Acceptance Testing)	Real, representative users trying the solution to confirm it is usable and understandable.
Defect	A confirmed gap between expected and actual behavior, tracked until resolved.
Sign-off criteria	The agreed conditions that must all be true before production deployment proceeds.
Regression	A previously working feature that breaks as a side effect of a later change — worth re-testing key flows after any fix.

16. Summary and Project Wrap-Up

Completing this phase closes out the seventh and final technical phase of the Employee Raise Issue project. With functional, security, UI/UX, and performance testing complete, defects resolved, and UAT and sign-off criteria met, the solution built across Phases 1–7 is ready for promotion to Production following your organization's standard change management process.

The full project documentation (covering Phases 1–8, including the project Conclusion, deployment, training, and support model) remains the reference for anything beyond this individual phase.

16.1 Phase 7 Deliverables Checklist

Deliverable	Status
Functional test cases executed and passing	Complete
Security testing executed, including direct API checks	Complete
UI/UX and cross-device testing executed	Complete
Performance testing executed under realistic data volume	Complete
Defects logged and resolved	Complete
UAT conducted with representative employees	Complete
Sign-off criteria met	Ready for Production promotion